C Programming - Assignment 11

Q.1 Binary Search in Descending Order

Problem Statement

Write a program to perform Binary Search on an array sorted in **descending order** and find the given element.

Concepts Used

- Arrays
- Binary Search
- Looping (`while`)
- Conditional Statements (`if`)
- Search Algorithm

Step-by-Step Approach

1. Accept the size of the array.
2. Accept array elements in descending order.
3. Accept the element to be searched.
4. Initialize `low = 0` and `high = n - 1`.
5. Find the middle element.
6. Compare the middle element with the search element.
7. Since the array is in descending order:
    - If the search element is greater than the middle element, search the left half.
    - If the search element is smaller than the middle element, search the right half.
8. Repeat until the element is found or the search range becomes invalid.
9. Display the position if found; otherwise display "Element Not Found".

Test Cases

Test Case	Array (Descending)	Search Element	Output
1	90 80 70 60 50 40 30	60	Position = 4
2	100 90 80 70 60	90	Position = 2
3	50 40 30 20 10	15	Element Not Found

Q.2 Linear Search to Find the Nth Occurrence

Problem Statement

Implement a Linear Search algorithm to find the **nth occurrence** of the given element. If the nth occurrence is not found, return `-1`.

Concepts Used

- Arrays
- Linear Search
- Looping (`for`)
- Conditional Statements (`if`)
- Counter Variable

Step-by-Step Approach

1. Accept the size of the array.
2. Accept the array elements.
3. Accept the element (`k`) to search.
4. Accept the occurrence number (`n`).
5. Traverse the array from beginning to end.
6. Whenever the element matches, increment the occurrence counter.
7. If the counter becomes equal to `n`, print the current index.
8. If the loop ends before finding the nth occurrence, print `-1`.

Test Cases

Test Case	Input	Output
1	arr = 1 2 3 1 1 3 4 1 2, k = 1, n = 2	3
2	arr = 1 2 3 1 1 3 4 1 2, k = 1, n = 4	7
3	arr = 1 2 3 1 1 3 4 1 2, k = 1, n = 5	-1

Q.3 Optimized Bubble Sort Using Flag Variable

Problem Statement

Implement the Bubble Sort algorithm using a **flag variable** so that the algorithm runs in **O(n)** time complexity in the best case.

Concepts Used

- Arrays
- Bubble Sort
- Nested Loops (`for`)
- Swapping Technique
- Flag Variable

- Conditional Statements (`if`)

Step-by-Step Approach

1. Accept the size of the array.
2. Accept the array elements.
3. Compare adjacent elements.
4. Swap the elements if they are in the wrong order.
5. Set the flag whenever a swap occurs.
6. After each pass, check the flag.
7. If no swapping occurred, terminate the algorithm because the array is already sorted.
8. Display the sorted array.

Test Cases

Test Case	Input	Output
1	5 4 3 2 1	1 2 3 4 5
2	1 2 3 4 5	1 2 3 4 5
3	10 20 15 30 25	10 15 20 25 30